

Backend Architecture Deep Dive - Interview Level Understanding

■ Table of Contents

1. Overall Architecture Philosophy
 2. Project Structure & Organization
 3. Core Components Explained
 4. Data Flow & Request Lifecycle
 5. Design Decisions & Why
 6. Alternatives We Considered
 7. How to Make Changes
-

Overall Architecture Philosophy

What Problem Does This Solve?

The backend is a **REST API + AI Agent** that answers cricket questions by:

1. Storing cricket match data in a database
2. Allowing both structured API calls and natural language AI queries
3. Combining **database tool** (SQL) with **AI reasoning** (OpenAI LLM)

This is a **Version 2 (V2)** implementation:

- **V1 was:** Data + basic API (just CRUD operations)
- **V2 is:** Data + API + AI agent that can read the database
- **V3 would be:** Add text retrieval (RAG with ChromaDB)
- **V4 would be:** Add intelligent routing between multiple tools (LangGraph)

Why FastAPI?

FastAPI is chosen because:

- ■ Fast async support (can handle multiple requests simultaneously)
 - ■ Automatic API documentation (Swagger UI at [/docs](#))
 - ■ Strong type hints and validation (Pydantic)
 - ■ Easy to integrate with LLMs and external APIs
 - ■ NOT Flask (too minimal for this complexity)
 - ■ NOT Django (too heavyweight, overkill for an API-first service)
-

Project Structure & Organization

```
Backend/
├── main.py                # FastAPI app entrypoint (registers all routes)
├── requirements.txt       # Python dependencies
├── cricket_ai.db          # SQLite database (the data)
├──
├── app/
│   ├── __init__.py
│   ├──
│   ├── core/              # *** Core infrastructure ***
│   │   ├── config.py      # Settings & environment variables
│   │   ├── database.py    # SQLAlchemy setup (connection + session factory)
│   │   ├──
│   │   ├── models/        # *** Database schema (ORM) ***
│   │   │   ├── match.py   # Match table definition
│   │   │   ├── delivery.py # Delivery (ball-by-ball) table definition
│   │   │   └── player.py  # Player table definition
│   │   ├──
│   │   ├── schemas/       # *** Request/Response validators ***
│   │   │   └── chat.py    # Pydantic models for chat endpoint
│   │   ├──
│   │   └── services/       # *** Business logic (optional) ***
│   │       └── (could add data processing here)
│   ├──
│   └── api/                # *** API routes/endpoints ***
│       ├── routes/
│       │   ├── health.py   # GET /api/health
│       │   ├── matches.py  # GET /api/matches, /api/matches/:id
│       │   ├── chat.py     # POST /api/chat (AI agent)
│       │   └── players.py  # GET /api/players/compare
│       ├──
│       └── tests/
│           ├── conftest.py  # Pytest setup
│           └── test_api.py  # Integration tests
```

Why This Structure?

| Folder | Why |

|-----|-----|

| **core/** | Centralize config & DB setup so all modules reuse the same database connection |

| **models/** | SQLAlchemy ORM models = database tables. Keeps schema definition separate from logic |

| **schemas/** | Pydantic validators ensure API requests are properly typed. Decouples API from DB |

| **api/routes/** | Each route file is a logical endpoint. Easy to find and modify |

| **tests/** | Integration tests verify all endpoints work together |

Core Components Explained

1. config.py - Settings Management

```
class Settings(BaseSettings):  
    DATABASE_URL: str = "sqlite:///./cricket_ai.db"  
    OPENAI_API_KEY: str | None = None
```

What it does:

- Reads **.env** file for configuration
- Provides a single **settings** object that all modules use
- Resolves SQLite paths dynamically (works from any directory)

Why this pattern?

- ■ Centralized config (no hardcoded values scattered in code)
- ■ Environment variables can override defaults (local dev vs. production)
- ■ **BaseSettings** auto-loads from **.env** file
- ■ Alternative: Pass config as arguments everywhere (tedious, error-prone)

How to change it:

Edit **.env** file to use PostgreSQL instead:

```
DATABASE_URL=postgresql://user:pass@localhost/cricket_ai
```

2. database.py - Database Connection

```
engine = create_engine(settings.DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

What it does:

- Creates a database engine (connection pool)
- Creates a session factory (for transactions)
- Provides `get_db()` dependency that FastAPI injects into routes

Why this pattern?

- ■ Single database connection reused across all requests
- ■ Automatic session cleanup (finally block)
- ■ FastAPI's dependency injection manages the session lifecycle
- ■ Alternative: Open/close connection per request (slower, wasteful)

How the `get_db()` injection works:

In matches.py route

```
@router.get("/matches/")
def get_all_matches(db: Session = Depends(get_db)): # FastAPI provides db here
    matches = db.query(Match).all()
    return matches
```

3. Models - Database Schema (ORM)

What is ORM?

ORM = Object-Relational Mapping. Instead of writing raw SQL, you define Python classes that map to database tables.

models/match.py

```
class Match(Base):
    __tablename__ = "matches"
    id = Column(Integer, primary_key=True)
    team_1 = Column(String(255))
    team_2 = Column(String(255))
    winner = Column(String(255))
    start_date = Column(Date)
```

Why ORM instead of raw SQL?

- ■ Type safety (Python catches errors before DB)
- ■ Relationships are explicit (e.g., **match.deliveries**)
- ■ Easy migrations (change schema programmatically)
- ■ Raw SQL is faster for complex queries (but we rarely need that)

The Three Tables:

| Table | What It Stores | Why |

|-----|-----|-----|

| **matches** | Match metadata (teams, date, venue, winner) | Core match info; few records |

| **deliveries** | Ball-by-ball data (batter, bowler, runs, wickets) | Granular stats; many records (1000s per match) |

| **players** | Player names & team | Could extend with more stats later |

4. Schemas - Request/Response Validation

schemas/chat.py

```
class ChatRequest(BaseModel):
    query: str
```

What it does:

- Defines expected structure of API requests
- Pydantic validates & converts types automatically
- Invalid requests are rejected with clear error messages

Why separate from models?

- Models = database tables
- Schemas = API contracts
- A request might need only 1 field, but the database has 10 fields
- Decoupling lets you evolve the API independently of the schema

Example:

Bad request (missing field)

Response: {"detail": "query is required"}

Good request

Response: {"answer": "..."}

```
POST /api/chat
{}
POST /api/chat
{"query": "How many runs did Babar score?"}
```

5. Routes/Endpoints - The API Surface

health.py - Liveness Check

```
@router.get("/")
def health_check():
```

```
return {"status": "healthy", "service": "backend"}
```

- **Purpose:** Frontend/monitoring checks if backend is alive
- **Why:** Kubernetes, load balancers, monitoring tools use this to detect failures

matches.py - Match Data

```
@router.get("/matches/")
def get_all_matches(page: int = 1, per_page: int = 12, team: str = None, db: Session = Depends(get_db)):
    query = db.query(Match)
    if team:
        query = query.filter((Match.team_1 == team) | (Match.team_2 == team))
    matches = query.limit(per_page).offset((page - 1) * per_page).all()
    return {"count": total, "matches": matches}
```

Design decisions:

- ■ Pagination (limit/offset) prevents huge responses
- ■ Filtering by team (allows frontend search)
- ■ Returns both total count and paginated results
- ■ Alternative: Return all matches (slow, memory-heavy)

chat.py - AI Agent

```
db = SQLiteDatabase.from_uri(settings.DATABASE_URL)
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
agent = create_sql_agent(llm, db, agent_type="openai-tools")
response = agent.invoke({"input": request.query})
```

How it works:

1. User sends: **{"query": "How many runs did Babar score?"}**
2. LangChain creates an agent that can:
 - Understand the question
 - Generate SQL queries
 - Execute them against the database
 - Interpret results
 - Return a human-readable answer
3. The **AGENTPROMPTPREFIX** tells the AI what tables/columns exist

Why this design?

- ■ AI agents are "reasoning engines" (not just completions)
- ■ OpenAI-tools can plan multi-step queries
- ■ Agents retry if a query fails
- ■ Alternative: Simple LLM call (would fail on complex questions)

players.py - Player Comparison

```
@router.get("/players/compare")
def compare_players(player1: str, player2: str, db: Session = Depends(get_db)):
    stats1 = compute_batting_stats(deliveries_for_player1)
    stats2 = compute_batting_stats(deliveries_for_player2)
    return {"player_1": stats1, "player_2": stats2}
```

Why a dedicated endpoint?

- ■ Structured data (always same format)
 - ■ Frontend can display comparison side-by-side
 - ■ Alternative: Ask AI agent (slower, less reliable)
-

Data Flow & Request Lifecycle

Scenario: Frontend asks "How many runs did Babar score?"

1. Frontend sends HTTP request
POST /api/chat
{ "query": "How many runs did Babar score?" }
2. FastAPI receives request in chat.py
3. Pydantic validates: { "query": "..." }
4. get_db() dependency provides a database session
5. LangChain agent starts:
 - a) Reads AGENT_PROMPT_PREFIX (tells AI about schema)
 - b) Passes user query to ChatOpenAI (gpt-4o-mini)
 - c) OpenAI generates SQL: "SELECT SUM(runs_batter) FROM deliveries WHERE batter='Babar'"
 - d) LangChain executes SQL against database
 - e) Database returns: [{"sum": 2847}]
 - f) OpenAI interprets: "Babar scored 2,847 runs"
6. Route returns response
{ "answer": "Babar scored 2,847 runs" }
7. Frontend displays in UI

Why Each Step Matters

Step	Why
Validation (Pydantic)	Catches bad input early; prevents SQL injection
get_db()	Reuses connection; manages lifecycle
AGENTPROMPTPREFIX	Constraints AI to available tables; improves accuracy
Temperature=0	Makes AI deterministic (always same answer)
Error handling	If SQL fails, user sees error instead of crash

Design Decisions & Why

Decision 1: SQLite vs PostgreSQL

We chose: SQLite (for now)

Feature	SQLite	PostgreSQL
Setup	No server needed	Separate server
File size	Small (single file)	Larger (but scalable)
Concurrency	Limited	Excellent
Use case	Development, small projects	Production, high load

When to switch to PostgreSQL:

- Deployment is happening
- Multiple users query simultaneously
- Need complex indexing for 1M+ records

How to switch:

.env file

(everything else stays the same!)

DATABASE_URL=postgresql://user:pass@localhost/cricket_ai

Decision 2: FastAPI + SQLAlchemy ORM

We chose: FastAPI + SQLAlchemy ORM

| Approach | Pros | Cons |

|-----|-----|-----|

| **FastAPI + ORM** | Type safe, automatic docs, easy relationships | Slower than raw SQL |

| **FastAPI + Raw SQL** | Fast, flexible | No validation, SQL injection risk |

| **Django ORM** | Batteries included | Heavyweight, slower startup |

| **Flask + SQLAlchemy** | Minimal | Requires more manual setup |

Decision rationale:

- Development speed > raw speed (cricket queries are not high-frequency)
 - Type safety prevents bugs
 - Auto-documentation helps frontend team
-

Decision 3: LangChain Agent Architecture

We chose: LangChain SQL Agent

```
agent = create_sql_agent(  
    llm=ChatOpenAI(model="gpt-4o-mini", temperature=0),  
    db=db,  
    agent_type="openai-tools"  
)
```

What an agent does:

- Can call multiple tools (SQL, web search, calculator)
- Plans multi-step sequences

- Retries if a step fails
- Interprets results

Alternative architectures:

| Approach | How It Works | Pros | Cons |

|-----|-----|-----|-----|

| **Agent (chosen)** | AI plans & executes SQL | Flexible, self-correcting | Slower (multiple LLM calls) |

| **Prompt-based** | AI generates SQL in one shot | Fast | Fails on complex questions |

| **Few-shot** | Show examples then ask AI | Good accuracy | Requires hand-crafted examples |

| **RAG** | Retrieve docs then answer | Factual, grounded | Need document database |

When to switch:

- If response time becomes critical → use prompt-based
- If data is very large → add RAG layer
- If need autonomous tool selection → use LangGraph

Decision 4: LLM Model Choice (gpt-4o-mini)

We chose: **gpt-4o-mini**

| Model | Cost | Speed | Quality |

|-----|-----|-----|-----|

| **gpt-4o-mini** | \$0.15 per 1M tokens | Fast | Good |

| gpt-4 | \$0.03 per 1K input | Slow | Best |

| gpt-3.5-turbo | \$0.50 per 1M tokens | Fast | Okay |

Decision rationale:

- Good enough for cricket questions (not ambiguous domain)
- 10x cheaper than gpt-4
- Fast enough for interactive chat

When to upgrade:

- If accuracy drops below acceptable threshold
- If more complex reasoning needed (V4 with LangGraph)

Alternatives We Considered

Alternative 1: No AI Agent

What it would look like:

```
@router.get("/players/{name}/stats")
def get_player_stats(name: str, db: Session = Depends(get_db)):
    stats = db.query(Delivery).filter(Delivery.batter == name).all()
    return compute_stats(stats)
```

Why we didn't:

- ■ Frontend has to know exact endpoints
- ■ Can't ask natural language questions
- ■ User experience is technical, not conversational
- ■ But it's faster and simpler

When you'd use this:

- Highly structured use cases (e.g., internal dashboards)
 - Performance is critical
 - Limited AI budget
-

Alternative 2: Custom SQL Generator

Instead of using LangChain agent

```
sql = generate_sql_from_query(user_query) # Custom code
result = db.execute(sql)
```

Why we didn't:

- ■ Need to hand-code SQL generator

- ■ Fails on questions we didn't anticipate
 - ■ More maintenance burden
 - ■ But completely customizable
-

Alternative 3: Vector Database (ChromaDB)

Load all match reports into embeddings

When user asks, search vectors + query database

```
vectors = encode_match_reports()  
store_in_chromadb(vectors)  
relevant_docs = chromadb.query(user_query)
```

Why we didn't (yet):

- ■ Requires match reports (we only have data)
- ■ More complex setup
- ■ Would enable "Why did Babar get out?" questions

When to add (V3):

- Have match reports/commentary
 - Need contextual answers beyond statistics
-

How to Make Changes

Common Changes & How To Do Them

1. Add a New Endpoint

Goal: Add **GET /api/teams** to list all teams

Steps:

1. Create new file **app/api/routes/teams.py**:

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from sqlalchemy import distinct
from app.core.database import get_db
from app.models.match import Match
router = APIRouter(prefix="/teams", tags=["Teams"])
@router.get("/")
def get_all_teams(db: Session = Depends(get_db)):
    teams = db.query(distinct(Match.team_1)).all()
    return {"teams": [t[0] for t in teams]}
```

2. Register in **main.py**:

```
from app.api.routes.teams import router as teams_router
app.include_router(teams_router, prefix="/api")
```

3. Test:

```
curl http://localhost:8000/api/teams
```

2. Change Database Schema (Add Column)

Goal: Add **toss_winner** column to Match table

Steps:

1. Edit **models/match.py**:

```
class Match(Base):
    __tablename__ = "matches"
    id = Column(Integer, primary_key=True)
    toss_winner = Column(String(255), nullable=True) # NEW
```

2. Update data loading script to populate **toss_winner**

3. Update API routes that return matches (they auto-include new field)

Note: SQLAlchemy uses Flask-Migrate or Alembic for schema versions in production. For dev, you can delete `cricket_ai.db` and restart.

3. Modify AI Agent Prompt

Goal: Make AI prioritize recent matches

Steps:

Edit `app/api/routes/chat.py`:

```
AGENT_PROMPT_PREFIX = """
...
When answering questions about player performance, prioritize matches
from the last 2 years (start_date > 2024-01-01).
...
"""
```

4. Add New Test

Goal: Test that `/api/players/compare` works

Steps:

Edit `tests/test_api.py`:

```
def test_player_compare_works():
    response = client.get("/api/players/compare?player1=Babar&player2=Virat")
    assert response.status_code == 200
    data = response.json()
    assert "player_1" in data
    assert "player_2" in data
    assert "batting" in data["player_1"]
```

Run: `pytest -q`

5. Switch Database (SQLite → PostgreSQL)

Steps:

1. Install PostgreSQL locally or on a server
2. Create database: **createdb cricket_ai**
3. Update **.env**:

```
DATABASE_URL=postgresql://user:password@localhost/cricket_ai
```

4. Install Python driver: **pip install psycopg2-binary**
 5. Restart backend—everything works!
-

Interview Questions You Should Be Able to Answer

1. "Why FastAPI over Flask?"

- Answer: Type safety, async support, automatic documentation, better for integrating with LLMs

2. "How does the AI agent avoid SQL injection?"

- Answer: SQLAlchemy parameterizes queries; LangChain's SQL agent uses verified dialects

3. "What would break if someone deletes cricket_ai.db?"

- Answer: All data lost; database recreates on next startup if using SQLAlchemy's **Base.metadata.create_all()**

4. "How does pagination work in **/api/matches?**"

- Answer: $\text{limit}=\text{perpage records}$, $\text{offset}=(\text{page}-1)*\text{perpage}$. Frontend requests page 1, 2, 3, etc.

5. "What happens if OpenAI API is down?"

- Answer: Chat endpoint fails with 500 error. Health check still works. Other endpoints unaffected.

6. "Why use an ORM instead of raw SQL?"

- Answer: Type safety, less boilerplate, auto-validation, easier schema changes

7. "What is the purpose of **getdb()** in **Depends(getdb)**?"

- Answer: FastAPI dependency injection. Provides a fresh DB session; closes it after request completes.

8. "Why temperature=0 for the LLM?"

- Answer: Deterministic output. Same query always gives same answer (no randomness).

9. "How would you add vector search (RAG)?"

- Answer: Load match reports into ChromaDB, query vectors for context, pass to AI agent

10. "What would happen if a user sends " OR "1"="1 as a query?"

- Answer: LangChain handles it safely; Pydantic validates it's a string; SQL is parameterized

Summary Table: Core Components

Component	File	Purpose	Key Design
Settings	<code>config.py</code>	Environment config	Centralized, <code>.env</code> -based
Database	<code>database.py</code>	Connection pool	SQLAlchemy engine + session factory
Models	<code>models/*.py</code>	ORM schema	Define tables as Python classes
Schemas	<code>schemas/*.py</code>	Request validation	Pydantic models for API contracts
Health	<code>routes/health.py</code>	Liveness check	Simple status endpoint
Matches	<code>routes/matches.py</code>	Match CRUD	Pagination + filtering
Chat	<code>routes/chat.py</code>	AI agent	LangChain SQL agent + OpenAI
Players	<code>routes/players.py</code>	Player stats	Dedicated comparison endpoint

This is your interview-level explanation. Save this document and reference it when making changes!